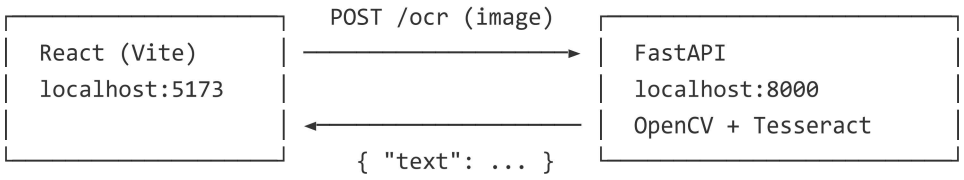
Build an Image-to-Text App with React, FastAPI and Tesseract

In this tutorial you will build a small web app:

1. You pick a picture in the browser (React).
2. The picture is sent to a Python server (FastAPI).
3. The server cleans the image with OpenCV and reads the text with Tesseract.
4. The text is sent back and shown on the page.



---

1. Prerequisites

You need:

- **Python 3.10+**
- **Node.js 18+** (includes `npm`)
- **Tesseract OCR** installed on your computer (this is a separate program, not just a Python package)

Install Tesseract with English, German and Swahili

Ubuntu / Debian

```
sudo apt update
sudo apt install tesseract-ocr tesseract-ocr-eng tesseract-ocr-deu tesseract-ocr-swa
```

macOS (Homebrew)

```
brew install tesseract tesseract-lang
```

Windows Download the installer from the UB Mannheim Tesseract page, and tick the extra languages (German, Swahili) during setup. Then add the install folder (e.g. `C:\Program Files\Tesseract-OCR`) to your PATH.

Check it works:

```
tesseract --version
tesseract --list-langs     # should include eng, deu, swa
```

---

2. Project layout

2. Project layout

```
ocr-app/
├── backend/
│   ├── main.py
│   └── requirements.txt
└── frontend/          (created by Vite in step 4)
```

Create the folders:

```
mkdir -p ocr-app/backend
cd ocr-app/backend
```

---

3. The backend (FastAPI)

3.1 Set up a virtual environment

```
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
```

3.2 `requirements.txt`

```
fastapi
uvicorn[standard]
python-multipart
opencv-python-headless
pytesseract
numpy
```

Install:

```
pip install -r requirements.txt
```

> `python-multipart` is required by FastAPI to receive file uploads.

3.3 `main.py`

Your original script read a file from disk with `cv2.imread`. On a server, the image arrives as bytes in memory, so we use `cv2.imdecode` instead. The processing steps (grayscale, threshold, OCR) stay the same.

```
import cv2
import numpy as np
import pytesseract
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from starlette.concurrency import run_in_threadpool

app = FastAPI(title="OCR API")
```

```
# Allow the React dev server to call this API from the browser
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)


def extract_text(image_bytes: bytes) -> str:
    """Your original script, adapted to work with bytes."""
    # Turn raw bytes into an OpenCV image
    array = np.frombuffer(image_bytes, dtype=np.uint8)
    image = cv2.imdecode(array, cv2.IMREAD_COLOR)
    if image is None:
        raise ValueError("Could not read the image")

    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # More black and white
    _, threshold = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)

    # Read the text
    return pytesseract.image_to_string(threshold, lang="eng+deu+swa")


@app.get("/")
def health_check():
    return {"status": "ok"}


@app.post("/ocr")
async def ocr(file: UploadFile = File(...)):
    # Basic validation: only accept images
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Please upload an image file.")

    image_bytes = await file.read()

    try:
        # OCR is slow and blocking, so run it in a worker thread
        text = await run_in_threadpool(extract_text, image_bytes)
    except ValueError as err:
        raise HTTPException(status_code=400, detail=str(err))

    return {"filename": file.filename, "text": text.strip()}
```

3.4 Run it

```
uvicorn main:app --reload
```

Open http://localhost:8000/docs. FastAPI generates an interactive page where you can try `POST /ocr` by

Open http://localhost:8000/docs. FastAPI generates an interactive page where you can try `POST /ocr` by uploading an image, with no frontend needed. Test the backend here first.

---

4. The frontend (React + Vite)

Open a **second terminal** in the `ocr-app` folder:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

Vite prints a URL, normally http://localhost:5173.

4.1 Replace `src/App.jsx`

```
import { useState } from "react";
import "./App.css";

const API_URL = "http://localhost:8000";

function App() {
  const [file, setFile] = useState(null);
  const [previewUrl, setPreviewUrl] = useState(null);
  const [text, setText] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  function handleFileChange(event) {
    const selected = event.target.files[0];
    if (!selected) return;

    setFile(selected);
    setPreviewUrl(URL.createObjectURL(selected)); // local preview
    setText("");
    setError("");
  }

  async function handleSubmit(event) {
    event.preventDefault();
    if (!file) return;

    setLoading(true);
    setError("");
    setText("");

    // Files are sent as multipart form data
    const formData = new FormData();
    formData.append("file", file); // "file" must match the FastAPI parameter name

    try {
      const response = await fetch(`${API_URL}/ocr`, {
```

```
        method: "POST",
        body: formData,
      });

      if (!response.ok) {
        const data = await response.json();
        throw new Error(data.detail || "Something went wrong");
      }

      const data = await response.json();
      setText(data.text || "(No text found)");
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="container">
      <h1>Image to Text</h1>

      <form onSubmit={handleSubmit}>
        <input type="file" accept="image/*" onChange={handleFileChange} />
        <button type="submit" disabled={!file || loading}>
          {loading ? "Reading..." : "Extract text"}
        </button>
      </form>

      {previewUrl && (
        <img src={previewUrl} alt="Selected upload" className="preview" />
      )}

      {error && <p className="error">{error}</p>}

      {text && (
        <>
          <h2>Extracted text</h2>
          <textarea value={text} readOnly rows={12} />
          <button onClick={() => navigator.clipboard.writeText(text)}>
            Copy
          </button>
        </>
      )}
    </div>
  );
}

export default App;
```

4.2 Replace `src/App.css`

```
.container {
  max-width: 640px;
```

```
  margin: 2rem auto;
  padding: 0 1rem;
  font-family: system-ui, sans-serif;
}

form {
  display: flex;
  gap: 0.75rem;
  align-items: center;
  flex-wrap: wrap;
}

button {
  padding: 0.5rem 1rem;
  cursor: pointer;
}

.preview {
  display: block;
  max-width: 100%;
  max-height: 300px;
  margin: 1rem 0;
  border: 1px solid #ccc;
  border-radius: 6px;
}

textarea {
  width: 100%;
  font-family: monospace;
  padding: 0.5rem;
  box-sizing: border-box;
  margin-bottom: 0.5rem;
}

.error {
  color: #b00020;
}
```

Also open `src/index.css` and delete its contents (or at least the `body { display: flex; ... }` rules Vite adds), so the page isn't squeezed to one side.

---

5. Try it

With both servers running:

Terminal	Folder	Command
1	`ocr-app/backend`	`uvicorn main:app --reload`
2	`ocr-app/frontend`	`npm run dev`

Go to http://localhost:5173, choose an image (like your `swahili.png`), click **Extract text**, and the text appears

below.

---

6. How it works

- `FormData` is how a browser sends files. FastAPI receives it as an `UploadFile`.
- **CORS**: the browser blocks requests between different origins (port 5173 and port 8000 count as different). The `CORSMiddleware` tells the browser the React app is allowed.
- `cv2.imdecode` replaces `cv2.imread` because the image is in memory, not on disk.
- `run_in_threadpool` keeps the server responsive: OCR can take a second or two, and this stops it from freezing other requests.
- `lang="eng+deu+swa"` tells Tesseract to try English, German and Swahili together.

---

7. Troubleshooting

Problem	Likely cause / fix
`TesseractNotFoundError`	Tesseract isn't installed or not on your PATH. On Windows, set `pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"` in `main.py`.
`Error opening data file ... swa.traineddata`	Language pack missing. Install `tesseract-ocr-swa` (Linux) or `tesseract-lang` (macOS).
Browser console shows a CORS error	The origin in `allow_origins` must exactly match the URL in your address bar.
`Form data requires "python-multipart"`	Run `pip install python-multipart`.
Empty or garbled text	The fixed threshold of `150` doesn't suit every image. See below.

---

8. Ideas to extend it

1. **Better thresholding.** A fixed value of 150 fails on dark or unevenly lit photos. Try Otsu's method, which picks the value automatically:

   ```
   _, threshold = cv2.threshold(
       gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
   )
   ```

2. **Language picker.** Add a dropdown in React, send the choice as a form field, and receive it in FastAPI with `lang: str = Form("eng")` (import `Form` from `fastapi`).
3. **Drag and drop** instead of a file button.
4. **Deploy.** Put the API URL in an environment variable (`import.meta.env.VITE_API_URL`) and update

4. **Deploy.** Put the API URL in an environment variable (`import.meta.env.VITE_API_URL`) and update `allow_origins` for your real domain.

Happy coding!